



UNIVERSITATEA DIN BUCUREȘTI

**FACULTATEA DE
MATEMATICĂ ȘI INFORMATICĂ**



SPECIALIZAREA INFORMATICĂ

Lucrare de licență

**PLATFORMĂ WEB SIGURĂ PENTRU
PLANIFICAREA, MONITORIZAREA ȘI ANALIZA
ANTRENAMENTELOR DE FORȚĂ**

Absolvent

[Prenume NUME]

Coordonator științific

[Titlul și numele coordonatorului]

București, iulie 2026

Rezumat

Lucrarea prezintă o platformă web pentru planificarea și monitorizarea antrenamentelor de forță, construită cu accent pe corectitudinea datelor și pe controlul accesului. Aplicația permite unui utilizator să își gestioneze exercițiile, rutinele, sălile și echipamentele, să construiască programe pe mai multe zile, să execute sesiuni live, să consulte istoricul și progresul, să publice sau să copieze programe dintr-un marketplace și să colaboreze, în mod controlat, cu un antrenor.

Problema centrală tratată este separarea dintre datele de planificare, care se pot modifica, și istoricul antrenamentelor, care trebuie să rămână stabil. La pornirea unei sesiuni, sistemul copiază înregistrările relevante din program, sală, exerciții, grupe musculare, rutine și echipamente în instantanee istorice. Astfel, o sesiune finalizată rămâne corectă chiar dacă ulterior utilizatorul editează sau șterge datele sursă. Pe lângă acest model de date, lucrarea descrie autentificarea cu JWT și rotația token-urilor de reîmprospătare, protecția față de accesul neautorizat la resurse (IDOR), autorizarea pe roluri și pe relații pentru modul de coaching, un analizor determinist al structurii programelor și un model de citire separat pentru căutare full-text cu OpenSearch.

Backend-ul este implementat în Spring Boot 3 / Java 21, cu PostgreSQL 16 ca sursă unică de adevăr și schema gestionată exclusiv prin Flyway. Frontend-ul este o aplicație React cu TypeScript. Căutarea este evaluată experimental printr-o comparație între OpenSearch și o variantă PostgreSQL construită corect, pe un set de date generat până la 2 milioane de documente. Rezultatele arată că PostgreSQL este suficient pentru colecții mici, dar că OpenSearch devine justificat la dimensiuni mai mari, mai ales pentru căutarea tolerantă la greșeli de tastare.

Abstract

This thesis presents a web platform for planning and tracking strength training, built with a strong focus on data correctness and access control. The application lets users manage exercises, routines, gyms and equipment, build multi-day training programs, run live workout sessions, inspect history and progress, publish or copy programs through a marketplace, and collaborate with a coach under explicit relationship-based access rules.

The central problem is the separation between editable planning data and historical workout data that must remain stable. When a workout session starts, the system copies the relevant program, gym, exercise, muscle-group, routine and equipment data into historical snapshots. As a result, a completed session remains correct even if the source data is later edited or deleted. Besides this data model, the thesis describes JWT authentication with refresh-token rotation, IDOR-resistant resource access, role-based and relationship-based authorization for coaching, a deterministic program-structure analyzer, and a separate OpenSearch read model for full-text search.

The backend is implemented in Spring Boot 3 / Java 21, with PostgreSQL 16 as the single source of truth and Flyway as the only schema-management mechanism. The frontend is a React and TypeScript single-page application. Search is evaluated empirically by comparing OpenSearch with a properly indexed PostgreSQL baseline over a generated corpus scaled up to 2 million documents. The results show that PostgreSQL is enough for small collections, while OpenSearch becomes justified at larger sizes, especially for typo-tolerant search.

Cuprins

1	Introducere	6
1.1	Tipul lucrării și încadrarea temei	6
1.2	Prezentarea generală a temei	6
1.3	Problema urmărită	6
1.4	Contribuția proprie	7
1.5	Structura lucrării	8
2	Preliminarii	8
2.1	Aplicații web și sursa de adevăr	8
2.2	Date de planificare și date istorice	8
2.3	Autentificare, autorizare și IDOR	9
2.4	Căutare full-text și modele de citire	9
2.5	Analiză deterministă a programelor	10
3	Contribuția proprie	10
3.1	Vedere de ansamblu asupra aplicației	10
3.2	Modelul de date și istoricul prin instanțanee	11
3.3	Constrângeri, ștergere logică și concurență	11
3.4	Securitate și controlul accesului	12
3.5	Funcționalități de planificare și execuție	12
3.6	Istoric, analize și marketplace	13
3.7	Analizorul de programe	13
3.8	Modul de coaching	13
3.9	Modelul de citire pentru căutare	14
3.10	Evaluarea OpenSearch vs. PostgreSQL	15
3.11	Testare și verificarea corectitudinii	15
4	Concluzii	16
4.1	Concluzii privind realizarea temei	16
4.2	Relevanța rezultatelor	16
4.3	Dezvoltări ulterioare	17
	Bibliografie	18
	Anexa 1 – Schema bazei de date	19
	Anexa 2 – Listări de cod reprezentative	19
	Anexa 3 – Rezultatele complete ale evaluării	21

Anexa 4 – Contractul API (selecție)	21
Anexa 5 – Configurație și rulare	23

Capitolul 1. Introducere

1.1 Tipul lucrării și încadrarea temei

Lucrarea de față este o lucrare de licență aplicativă, încadrată în zona aplicațiilor web full-stack și a sistemelor de gestiune a datelor pentru domeniul fitness. Tema nu constă doar într-un jurnal de antrenamente, ci într-o platformă completă în care utilizatorul își poate planifica antrenamentele, le poate executa, le poate analiza după ce au fost realizate și poate descoperi programe create de alți utilizatori.

Din punct de vedere tehnic, proiectul combină mai multe subprobleme care apar într-un sistem real: autentificare sigură, autorizare pe resurse, date istorice care nu trebuie alterate, operații concurente, căutare full-text, marketplace, analiză automată a structurii unui program și acces controlat pentru antrenori. Tocmai această combinație este partea interesantă: fiecare funcționalitate trebuie să fie utilă pentru utilizator, dar și corectă în raport cu restul sistemului.

1.2 Prezentarea generală a temei

Un utilizator al aplicației începe prin a-și construi biblioteca de lucru: exerciții oficiale și exerciții proprii, rutine de încălzire sau încheiere, săli și echipamente. Pe baza acestora creează programe de antrenament împărțite pe zile. O zi de program conține exerciții ordonate, valori planificate (serii, repetări, greutate, pauză) și, opțional, rutine atașate.

În momentul în care utilizatorul pornește un antrenament, programul devine o sesiune live. Aici apar datele istorice propriu-zise: seriile efectuate, greutatea, repetările, durata, RPE-ul, notele și echipamentul folosit. După finalizare, sesiunea intră în istoric și este folosită în analizele de progres. Sistemul calculează volum în timp, antrenamente pe săptămână, distribuția pe grupe musculare și estimări de progres la forță.

Pe lângă fluxul individual, aplicația are un marketplace de programe publice. Un utilizator poate publica un program, iar ceilalți îl pot vota, salva sau copia în biblioteca lor privată. Există și un mod de coaching: un utilizator cu rol de antrenor poate vedea istoricul și analizele unui client numai dacă există o relație activă acceptată de client. Acest acces este intenționat limitat la citire, ca să nu amestece colaborarea cu dreptul de a modifica datele personale ale clientului.

1.3 Problema urmărită

Într-o aplicație de acest tip, problema cea mai importantă nu este doar să se poată salva antrenamente, ci să se poată avea încredere în ce s-a salvat. Datele de planificare sunt, prin natura lor, editabile: un utilizator poate redenumi un exercițiu, poate modifica un program, poate șterge o sală sau poate schimba echipamentul disponibil. În schimb, istoricul trebuie să

descrie ce s-a întâmplat la momentul respectiv. Dacă o sesiune veche s-ar schimba pentru că programul original a fost editat, aplicația ar pierde sensul de jurnal.

De aici rezultă prima direcție a lucrării: un model de date cu instantanee pentru istoricul antrenamentelor. La pornirea sesiunii, aplicația copiază datele relevante din planificare în tabele de sesiune. Cheile către datele sursă rămân doar ca legături auxiliare și sunt anulabile, în timp ce valorile afișate în istoric sunt cele copiate în momentul execuției.

A doua direcție este securitatea. Aplicația este multi-user și include resurse personale, relații antrenor-client și endpoint-uri administrative. De aceea, fiecare operație trebuie să știe cine este utilizatorul curent și ce are voie să vadă sau să modifice. Sistemul folosește JWT, token-uri de reîmprospătare opace rotite, interogări scoping după proprietar și reguli de acces care întorc 404 pentru resursele altui utilizator, pentru a evita scurgerile de informație de tip IDOR.

A treia direcție este regăsirea informației. Pe măsură ce crește numărul de programe și sesiuni, utilizatorul are nevoie de căutare după exerciții, grupe musculare, săli, echipamente, note și filtre. În proiect, căutarea este implementată cu OpenSearch ca model de citire derivat din PostgreSQL, dar lucrarea nu tratează OpenSearch ca soluție implicită. Ea include și o comparație practică împotriva unei variante PostgreSQL bine indexate, pentru a vedea când merită complexitatea unui motor separat.

1.4 Contribuția proprie

Contribuția proprie constă în proiectarea și implementarea unei platforme funcționale, nu doar a unui prototip izolat. Principalele elemente realizate sunt:

- un backend Spring Boot organizat pe funcționalități, cu separare între domeniu, servicii, repository-uri și controllere;
- o schemă PostgreSQL gestionată prin Flyway, cu constrângeri de integritate, indecși unici parțiali, ștergere logică și reguli de concurență la nivel de bază de date;
- un model de sesiuni cu instantanee, astfel încât istoricul rămâne corect după editarea sau ștergerea datelor de planificare;
- autentificare cu JWT, rotația token-urilor de reîmprospătare și protecție față de accesul neautorizat la resursele altor utilizatori;
- funcționalități complete pentru exerciții, rutine, săli, echipamente, programe, sesiuni live, istoric, analize, marketplace și coaching;
- un analizor determinist al structurii programelor, bazat pe reguli explicabile, fără a pretinde că oferă sfat medical;

- un model de citire OpenSearch pentru căutare full-text peste programe și istoric, cu filtre, fațete, evidențiere, toleranță la greșeli și verificare suplimentară în PostgreSQL;
- o evaluare experimentală OpenSearch vs. PostgreSQL pe un set de date generat până la două milioane de documente.

1.5 Structura lucrării

Capitolul 2 introduce noțiunile folosite în proiect: aplicații web, modelarea datelor istorice, autentificarea cu token-uri, controlul accesului, căutarea full-text și principiile folosite de analizorul de programe. Capitolul 3 descrie implementarea efectivă a platformei, urmărind fluxul de utilizare și apoi deciziile tehnice: modelul de date, securitatea, funcționalitățile, coaching-ul, căutarea, benchmark-ul și testarea. Capitolul 4 rezumă rezultatele și discută direcțiile de dezvoltare rămase. Detaliile care ar încărcă textul principal, cum ar fi schema bazei de date, fragmente de cod, tabelele complete de benchmark și contractul API, sunt incluse în anexe.

Capitolul 2. Preliminarii

În acest capitol prezint noțiunile tehnice pe care se sprijină proiectul. Le includ aici nu ca o listă de tehnologii, ci ca fundal pentru deciziile din implementare: de ce PostgreSQL rămâne sursa de adevăr, de ce istoricul se păstrează prin instantanee, de ce accesul trebuie verificat pe fiecare resursă și de ce căutarea este tratată ca model de citire separat.

2.1 Aplicații web și sursa de adevăr

Platforma este o aplicație web cu frontend React și backend Spring Boot. Frontend-ul trimite cereri JSON către un API REST, iar backend-ul aplică regulile de domeniu și persistă datele. Într-un astfel sistem, baza de date relațională nu este doar un loc de stocare, ci și componenta care poate garanta integritatea prin chei externe, constrângeri și tranzacții.

În proiect, PostgreSQL este sursa unică de adevăr. Această alegere este importantă deoarece aplicația are date care nu trebuie pierdute sau interpretate diferit: sesiuni de antrenament, token-uri de reîmprospătare, relații antrenor-client, voturi și salvări în marketplace. Hibernate este folosit pentru maparea obiect-relațională, dar schema nu este generată automat de aplicație. Ea este creată și modificată numai prin migrații Flyway, iar configurația Hibernate este setată pe `ddl-auto: validate`. Astfel, aplicația verifică schema la pornire, dar nu o schimbă implicit.

2.2 Date de planificare și date istorice

O diferență esențială în domeniul aplicației este cea dintre date de planificare și date istorice. Datele de planificare descriu intenția utilizatorului: programe, zile, exerciții planificate, rutine,

săli și echipamente. Ele trebuie să rămână editabile. Datele istorice descriu însă ce s-a întâmplat la un moment concret și nu ar trebui să se schimbe retroactiv.

Soluția folosită este modelarea prin instantanee. Un instantaneu nu înseamnă o copie a întregii baze de date, ci copierea atributelor relevante într-o entitate istorică. De exemplu, o sesiune nu se bazează pe numele curent al exercițiului, ci păstrează numele exercițiului copiat atunci când sesiunea a fost pornită. Același principiu se aplică grupelor musculare, programului, zilei, rutinei, sălii și echipamentului. Cheile externe către sursă pot exista în continuare pentru trasabilitate, dar trebuie să poată deveni NULL fără a strica istoricul.

2.3 Autentificare, autorizare și IDOR

Autentificarea stabilește identitatea utilizatorului care trimite o cerere. În aplicațiile moderne, un mod uzual de lucru este folosirea token-urilor JWT [2]: serverul semnează un token de acces, iar clientul îl trimite în antetul *Authorization*. Token-ul de acces trebuie să aibă o durată scurtă, deoarece este prezent la fiecare cerere. Pentru sesiuni mai lungi se folosește un token de reîmprospătare, care trebuie protejat mai strict.

Autorizarea răspunde la o altă întrebare: utilizatorul autentificat are voie să acceseze resursa cerută? Într-un sistem multi-user, o greșeală frecventă este IDOR (*Insecure Direct Object Reference*), adică situația în care un utilizator schimbă un identificator într-o cerere și obține datele altcuiva. O protecție corectă nu se bazează pe identificatori primiți de la frontend, ci pe utilizatorul curent extras din token și pe interogări care includ proprietarul resursei.

Pe lângă regulile obișnuite pe roluri (RBAC), modul de coaching cere și reguli bazate pe relații (ReBAC). Un utilizator cu rol de antrenor nu ar trebui să poată vedea orice utilizator, ci doar clienții cu care are o relație activă. De aceea, rolul este doar primul filtru; al doilea este relația antrenor-client acceptată.

2.4 Căutare full-text și modele de citire

PostgreSQL poate face căutare full-text prin *tsvector*, indecși GIN și funcții de ranking [5]. Pentru colecții mici sau pentru interogări simple, aceasta poate fi o soluție suficientă și mai ușor de operat decât un motor separat. Când apar însă cerințe precum toleranță la greșeli de tastare, sinonime, scoruri de relevanță pe mai multe câmpuri, evidențierea fragmentelor și fațete, un motor dedicat precum OpenSearch [3] devine mai potrivit.

În această lucrare, OpenSearch nu înlocuiește PostgreSQL. El este un model de citire derivat: documentele indexate sunt reconstruibile din baza de date relațională, iar rezultatele sunt validate din nou în PostgreSQL înainte de a fi returnate. Această abordare introduce consistență eventuală, dar păstrează o sursă unică de adevăr și permite refacerea indexului în caz de eroare.

2.5 Analiză deterministă a programelor

Aplicația include și un analizor al structurii programelor. Scopul lui nu este să ofere recomandări medicale și nici să înlocuiască un antrenor, ci să dea un feedback explicabil despre structură: volumul săptămânal, acoperirea grupelor musculare, frecvența, echilibrul între regiuni, lungimea sesiunilor și pauzele. Regulile sunt inspirate din literatura despre volumul antrenamentului [4] și din formule simple pentru estimarea progresului, cum este formula lui Epley pentru 1RM [1].

Am ales o abordare deterministă, bazată pe scăderi fixe dintr-un scor, deoarece este mai ușor de explicat și de testat decât o soluție de tip „AI coach”. Pentru o lucrare de licență și pentru un produs care atinge antrenamentul fizic, această onestitate este importantă: sistemul poate semnaliza probleme de structură, dar nu poate cunoaște recuperarea, tehnica, somnul, alimentația sau nivelul real de efort al utilizatorului.

Capitolul 3. Contribuția proprie

3.1 Vedere de ansamblu asupra aplicației

Platforma implementată urmărește întregul ciclu al unui antrenament. Utilizatorul își definește mai întâi datele de bază: exerciții, grupe musculare, rutine, săli și echipamente. Apoi construiește programe pe mai multe zile. O zi de program poate conține exerciții planificate, valori țintă și rutine de început sau final. Dintr-o astfel de zi se pornește o sesiune live, în care se adaugă seriile efectuate și notele din timpul antrenamentului. După finalizare, sesiunea devine istoric și intră în paginile de progres, căutare și coaching.

Aplicația nu este construită ca un CRUD simplu. Fiecare zonă are reguli de domeniu: un utilizator nu poate porni două antrenamente active în același timp; un program public trebuie să aibă o dată de publicare; un antrenor nu poate deveni propriul client; o sesiune finalizată nu mai permite modificarea seriilor; un program copiat din marketplace devine o copie privată independentă. Aceste reguli sunt implementate atât în servicii, cât și, unde este posibil, în baza de date.

Backend-ul este organizat pe funcționalități: auth, exercise, routine, gym, template, session, history, analytics, marketplace, analyzer, search, coaching și shared. În interiorul fiecărei funcționalități se păstrează aceeași separare: model de domeniu, servicii de aplicație, repository-uri și controllere. Frontend-ul React urmează aceeași împărțire pe funcționalități și consumă API-ul prin DTO-uri, fără să depindă de entitățile JPA.

O selecție a contractului API este prezentată în Anexa 4. Ea arată cum sunt grupate fluxurile principale: autentificare, planificare, sesiuni live, istoric, marketplace, analizor, căutare și coaching.

3.2 Modelul de date și istoricul prin instantanee

Schema este creată prin migrarea Flyway V1, care conține 28 de tabele. Migrarea V2 adaugă grupele musculare și exercițiile oficiale, iar V3 introduce un index unic parțial pentru regula „un singur antrenament activ per utilizator”. O sinteză a tabelor și a constrângerilor se găsește în Anexa 1.

Modelul de date este împărțit în două zone. Prima zonă este cea de planificare: `workout_templates`, `template_days`, `template_day_exercises`, rutine, săli și echipamente. A doua zonă este cea istorică: `workout_sessions`, `session_exercises`, `session_routines`, `workout_sets`. Regula principală este că seriile efectuate nu sunt legate direct de exercițiile dintr-un template, ci de exercițiile copiate în sesiune:

```
workout_sets -> session_exercises -> workout_sessions
```

La pornirea unui antrenament, serviciul de sesiuni încarcă ziua de program a utilizatorului și creează o sesiune nouă într-o tranzacție. Pentru fiecare exercițiu planificat se creează un `session_exercise` cu numele exercițiului, tipul, poziția, seriile planificate, repetările, greutatea, pauza și notele copiate. Grupele musculare sunt copiate separat în `session_exercise_muscle_groups`. Rutinele sunt copiate în `session_routines`. Dacă în timpul antrenamentului se folosește un echipament, seria păstrează și `equipment_name_snapshot`.

Acest design face ca istoricul să fie stabil. Dacă după două săptămâni utilizatorul redenumeste un exercițiu, șterge echipamentul sau modifică programul, sesiunea veche nu se schimbă. Cheile externe către datele sursă sunt `ON DELETE SET NULL`, iar valorile folosite pentru afișarea istoricului sunt coloanele `*_snapshot`. Același principiu apare și la copierea programelor din marketplace: copia păstrează nume, tipuri și grupe musculare, iar legăturile către exerciții sunt păstrate doar când exercițiul este oficial și sigur de referit.

3.3 Constrângeri, ștergere logică și concurență

Am încercat să mut cât mai multe reguli stabile în PostgreSQL, nu doar în cod Java. Exemplele cele mai importante sunt constrângerile `CHECK` pentru enumerări, `days_per_week BETWEEN 1 AND 7`, `rpe BETWEEN 1.0 AND 10.0`, `finished_at >= started_at`, contoare nenegative și interdicția ca un antrenor să fie propriul client. Pentru programe publice există o constrângere care cere ca `published_at` să fie nenul atunci când `visibility='PUBLIC'`.

Resursele de planificare folosesc ștergere logică prin `deleted_at`. Astfel, o ștergere nu rupe istoricul care referă acea resursă. Pentru a permite reutilizarea numelor după ștergere, unicitatea este impusă prin indecși unici parțiali care iau în calcul doar rândurile active. De exemplu, un utilizator poate crea din nou o sală cu același nume după ce vechea sală a fost ștearsă logic.

Concurența este tratată explicit în două locuri. Pentru sesiunile live, indexul `ux_workout_sessions_one_active_per_user` garantează la nivel de bază de date că un utilizator nu poate

avea două sesiuni IN_PROGRESS. Serviciul verifică regula înainte, pentru un răspuns mai prietenos, dar garanția finală este indexul. Pentru marketplace, contoarele de voturi și salvări sunt actualizate prin SQL atomic și mărginit cu GREATEST(0, ...), evitând valori negative sau curse de tip citire-modificare-scriere.

3.4 Securitate și controlul accesului

Autentificarea este bazată pe token-uri JWT. La login sau înregistrare, backend-ul emite un token de acces semnat HS256, cu durată scurtă, și un token de reîmprospătare opac. Token-ul de acces este trimis de frontend în antetul Authorization; token-ul de reîmprospătare este transmis ca un cookie HttpOnly, SameSite=Strict, limitat la calea /api/auth. În baza de date se păstrează doar amprenta SHA-256 a token-ului de reîmprospătare.

La fiecare reîmprospătare, token-ul vechi este revocat și se emite o pereche nouă în aceeași tranzacție. Dacă un token furat este refolosit, amprenta lui există, dar rândul este deja revocat, iar cererea este respinsă. Parolele sunt stocate cu BCrypt. Configurația de securitate este stateless: rutele de autentificare și /api/health sunt publice, /api/admin/** cere rolul ADMIN, /api/coach/** cere rolul COACH, iar restul rutelor /api/** cer un utilizator autentificat.

Protecția față de IDOR este aplicată prin modul în care sunt încărcate resursele. Controllerele nu primesc userId din corpul cererii, ci folosesc utilizatorul curent extras din JWT. Serviciile încarcă resursele prin metode care includ proprietarul, iar resursele imbricate verifică lanțul de proprietate până la rădăcină. Dacă un utilizator cere o resursă care aparține altcuiva, răspunsul este 404, nu 403, pentru a nu confirma existența acelei resurse.

3.5 Funcționalități de planificare și execuție

Funcționalitățile de planificare sunt gândite ca blocuri reutilizabile. Catalogul de exerciții conține exerciții oficiale, vizibile tuturor, și exerciții custom, vizibile doar proprietarului. Fiecare exercițiu are grupe musculare primare sau secundare. Rutinele sunt texte reutilizabile pentru început sau final de antrenament. Sălile și echipamentele sunt personale, iar echipamentele sunt legate de o sală.

Un program de antrenament este compus din zile. Fiecare zi are exerciții ordonate, valori planificate și rutine atașate. După modificarea exercițiilor unei zile, backend-ul recalculează în PostgreSQL vectori agregați ai programului: grupe musculare, identificatori de exerciții oficiale și nume de exerciții. Acești vectori sunt utili pentru filtrare și căutare.

Execuția live pornește dintr-o zi de program și creează instantaneele descrise mai sus. În timpul sesiunii, utilizatorul poate adăuga serii, poate introduce greutate, repetări, durată, distanță, RPE, note și echipament, sau poate adăuga exerciții neplanificate. După finalizare sau anulare, sesiunea devine terminală. Seriile nu mai pot fi modificate după finalizare, dar notele sesiunii pot fi editate, fiind comentariu al utilizatorului, nu datele efective ale execuției.

3.6 Istoric, analize și marketplace

Istoricul afișează sesiunile utilizatorului, ordonate descrescător după data de început. Pentru a evita problema N+1, sumarul unei pagini (număr de exerciții, număr de serii și volum total) este calculat printr-o singură interogare în lot. Analizele de progres sunt calculate în PostgreSQL peste sesiunile finalizate: volum pe zi, antrenamente pe săptămână, distribuția seriilor după grupa musculară primară, cele mai bune serii pe exercițiu și evoluția estimării 1RM folosind formula lui Epley.

Marketplace-ul permite publicarea programelor private, dar numai dacă programul nu este gol. Un utilizator poate naviga programele publice, le poate sorta după nouitate, scor sau tendință, le poate vota, salva și copia. Copierea este o operație de tip deep copy: se creează un program privat nou, cu zile, exerciții, rutine și instantanee proprii. Nu se expun date private ale autorului, iar copia poate fi folosită independent de original.

Pentru interacțiuni, votul propriu asupra propriului program este respins, salvarea este idempotentă, iar contoarele sunt menținute în `template_stats`. Evenimentele de folosire sunt păstrate în `template_use_events`, astfel încât popularitatea programelor poate fi folosită la sortare și căutare.

3.7 Analizorul de programe

Analizorul este expus prin `GET /api/templates/{id}/analysis`. El citește structura programului dintr-un model de aplicație, nu din DTO-uri web, și produce un scor între 0 și 100. Scorul este împărțit în cinci sub-scoruri: volum și acoperire, frecvență, echilibru, designul sesiunii și specificitate / pauză. Fiecare sub-scor pornește de la maxim și scade penalizări fixe pentru problemele găsite.

Volumul este calculat doar pentru exercițiile de forță. Seriile sunt ponderate în funcție de rolul muscular: o grupă primară primește 1.0, iar una secundară 0.5. Exercițiile fără serii planificate nu primesc valori inventate; ele sunt excluse din volum și marcate ca date incomplete. Categoriile rezultate sunt `WELL_STRUCTURED` pentru scoruri de cel puțin 80, `DECENT_STRUCTURE` pentru scoruri de cel puțin 55 și `NEEDS_REVIEW` pentru restul.

Răspunsul include avertismente cu severitate, explicații și sugestii. Exemple de reguli sunt lipsa unei regiuni importante, volum prea mic sau excesiv, dezechilibru push/pull, dezechilibru lower/upper, sesiuni prea lungi sau pauze prea scurte. Analizorul include și limitări explicite: nu vede tehnica, recuperarea, somnul, alimentația sau efortul real, deci nu oferă sfat medical.

3.8 Modul de coaching

Modul de coaching combină RBAC și ReBAC. Rutele `/api/coach/**` cer rolul `COACH`, dar acest rol nu este suficient pentru a vedea datele unui client. Pentru fiecare citire legată de un

client, serviciul verifică existența unei relații active între antrenor și client. Dacă relația lipsește sau nu este activă, răspunsul este 404.

Relațiile au un ciclu de viață clar: PENDING, ACTIVE, REJECTED și REVOKED. Antrenorul trimite invitația, clientul o acceptă sau o respinge, iar oricare dintre părți poate revoca accesul. O constrângere din baza de date interzice relația cu sine însuși, iar un index unic parțial permite o singură relație vie pentru aceeași pereche.

Accesul antrenorului este doar de citire: lista de clienți activi, istoricul clientului, analizele, detaliile unei sesiuni și căutarea în istoricul clientului. Nu există endpoint prin care antrenorul să modifice sesiunile sau programele clientului. Această limitare este intenționată și menține separația dintre colaborare și proprietatea datelor.

3.9 Modelul de citire pentru căutare

Căutarea este implementată cu OpenSearch ca model de citire derivat. Există două indexuri logice: `templates` pentru programe și `workout_sessions` pentru istoricul antrenamentelor. Documentele sunt denormalizate și conțin câmpurile utile pentru căutare: nume, descrieri, exerciții, grupe musculare, săli, echipamente, note, scor de analiză și statistici de popularitate.

Interogările folosesc `multi_match` cu ponderi diferite pe câmpuri. De exemplu, numele programului contează mai mult decât descrierea, iar în istoricul de antrenament numele exercițiului contează mai mult decât nota sesiunii. Pentru greșeli de tastare se folosește `fuzziness=AUTO`, dar nu pentru interogări foarte scurte, pentru a evita potriviri zgomotoase. Analizorul de căutare include sinonime precum *pecs* -> *chest* sau *ohp* -> *overhead press*. Rezultatele pot include fațete și evidențieri cu `<mark>`.

Scrierea în index se face prin evenimente Spring declanșate după commit-ul tranzacției, pentru ca datele anulate prin rollback să nu fie indexate. Există două tipuri de actualizare: structurală, când se reconstruiește documentul complet, și de statistici, când se actualizează doar contoarele de popularitate. Dacă o scriere în OpenSearch eșuează, eroarea este jurnalizată, iar PostgreSQL rămâne sursa de adevăr.

Securitatea căutării are două straturi. Primul este un filtru obligatoriu derivat din utilizatorul curent: `marketplace` înseamnă `visibility=PUBLIC`, programele personale înseamnă `ownerUserId=apelant`, iar istoricul înseamnă `ownerUserId=apelant`. Al doilea strat este validarea rezultatelor în PostgreSQL. Identificatorii întorși de OpenSearch sunt verificați din nou în baza relațională, astfel încât un rezultat învechit, șters sau depus publicat nu ajunge la client.

Reconstrucția indexului creează un index nou, îl umple din PostgreSQL și apoi schimbă atomic alias-ul. Endpoint-ul manual de reindexare este permis doar rolului ADMIN. Dacă modulul de căutare este dezactivat sau OpenSearch nu răspunde, API-ul întoarce 503, iar interfața poate reveni la lista SQL autoritativă.

3.10 Evaluarea OpenSearch vs. PostgreSQL

Pentru a evita o concluzie bazată doar pe preferința pentru un motor de căutare, am implementat o evaluare practică între OpenSearch și PostgreSQL. Varianta PostgreSQL nu este una slabă: folosește `tsvector` ponderat, index GIN, `ts_rank_cd`, `pg_trgm` pentru fuzzy search și `GROUP BY` pentru fațete. Varianta OpenSearch folosește mapările și interogările aplicației.

Setul de date este generat determinist și crește de la 1.000 la 2.000.000 de documente. Am măsurat cinci scenarii: două căutări full-text, o căutare fuzzy cu greșeală de tastare, o căutare filtrată și o agregare pe fațete. Tabelul 3.1 arată rezultatele la dimensiunea maximă, iar tabelele complete sunt în Anexa 3.

Scenariu la 2M documente	PostgreSQL	OpenSearch
Full-text „bench press” (p_{50})	1864 ms	64 ms
Full-text „squat” (p_{50})	1437 ms	4 ms
Fuzzy „benhc” (p_{50})	15825 ms	4 ms
Filtrat (press + INTERMEDIATE + PPL) (p_{50})	2295 ms	7 ms
Fațetă după dificultate (p_{50})	1193 ms	2 ms
Timp de construcție a indexului	239 s	115 s
Dimensiune pe disc	2632 MB	739 MB

Tabela 3.1: Comparație OpenSearch vs. PostgreSQL la 2.000.000 de documente.

Rezultatul principal este că cele două soluții se echivalează în jurul a 10.000 de documente. Sub acest prag, PostgreSQL este mai simplu și uneori mai rapid. Peste el, OpenSearch câștigă clar, mai ales la căutarea tolerantă la greșeli de tastare. Aceasta justifică arhitectura cu model de citire separat, dar și arată că un motor separat nu trebuie introdus fără un motiv concret.

3.11 Testare și verificarea corectitudinii

Suita de teste este gândită să ruleze pe infrastructură reală. Testele de integrare pornesc PostgreSQL 16 prin Testcontainers, iar testele de căutare pornesc și un container OpenSearch și activează modulul de search. Nu se folosește H2 și nu sunt înlocuite repository-urile cu implementări false.

Testele acoperă fluxuri HTTP, reguli de securitate, IDOR, marketplace, coaching, analizoare, sesiuni, istoric, căutare și garanții ale bazei de date. Unele teste verifică explicit constrângerile din PostgreSQL, de exemplu respingerea unei a doua sesiuni active inserate direct sau stabilitatea instantaneelor după editarea datelor sursă. În forma curentă, suita conține 128 de teste automate și cere un mediu Docker funcțional pentru verificările bazate pe Testcontainers.

Fragmente reprezentative din implementarea analizorului, din configurația de securitate și din migrările SQL sunt incluse în Anexa 2. Configurația de rulare a aplicației este rezumată în Anexa 5.

Capitolul 4. Concluzii

4.1 Concluzii privind realizarea temei

Lucrarea a pornit de la ideea unei aplicații de fitness, dar rezultatul este mai apropiat de o platformă completă pentru planificarea și urmărirea antrenamentelor de forță. Utilizatorul poate construi programe, porni sesiuni live, salva istoricul, urmări progresul, publica programe în marketplace, primi o analiză structurală a programului și, dacă dorește, poate colabora cu un antrenor. Aceste funcționalități sunt legate între ele prin același principiu: datele editabile rămân separate de istoricul executat.

Din punct de vedere tehnic, partea cea mai importantă a proiectului este modul în care funcționalitățile sunt protejate și păstrate corecte. Instantaneele fac ca o sesiune finalizată să rămână fidelă momentului în care a fost executată, chiar dacă ulterior se schimbă exercițiile, echipamentul sau programul inițial. Autentificarea cu JWT și token-uri de reîmprospătare rotite, verificările de proprietate, protecția față de IDOR și combinația RBAC/ReBAC din modulul de coaching limitează accesul la date fără să se bazeze pe identificatori trimiși de frontend. În plus, regulile stabile sunt dublate de constrângeri în PostgreSQL, nu doar de verificări în servicii.

Aplicația include și componente mai avansate, dar acestea sunt integrate în povestea domeniului. Analizorul de programe oferă feedback determinist și explicabil, nu un sfat opac. Marketplace-ul folosește copii independente, astfel încât un program public poate fi reutilizat fără să rupă datele autorului. Căutarea cu OpenSearch este implementată ca model de citire derivat din PostgreSQL, iar rezultatele sunt validate din nou în baza relațională. Benchmark-ul arată că PostgreSQL este suficient pentru colecții mici, dar că un motor dedicat devine justificat la volume mari, mai ales pentru căutarea tolerantă la greșeli de tastare.

4.2 Relevanța rezultatelor

Consider că valoarea lucrării vine din faptul că proiectul nu demonstrează o singură tehnologie izolată, ci o arhitectură coerentă. Fiecare zonă importantă are o regulă clară: PostgreSQL este sursa de adevăr, Flyway controlează schema, sesiunile finalizate sunt istorice, accesul este mereu verificat în funcție de utilizatorul curent, iar indexul de căutare poate fi reconstruit. Acest tip de proiectare este important pentru aplicații reale, unde o funcționalitate aparent simplă poate deveni problematică dacă istoricul se modifică retroactiv sau dacă un utilizator poate vedea datele altcuiva schimbând un ID.

Rezultatele sunt transferabile și în afara domeniului fitness. Separarea dintre date editabile și instantanee istorice apare și în comerț electronic, programări, facturare sau documente. Modelul de citire pentru căutare se aplică oricărui sistem în care datele relaționale trebuie găsite rapid prin text și filtre. Regulile de acces pentru coaching seamănă cu multe situații în care un utilizator

are acces delegat, dar limitat, la datele altuia.

4.3 Dezvoltări ulterioare

Există mai multe direcții naturale de dezvoltare. Prima este folosirea efectivă a tabelii `outbox_events` pentru un outbox tranzacțional, astfel încât indexarea în OpenSearch să fie reluabilă și auditabilă după erori. A doua este adăugarea unui cache Redis pentru clasamentele din marketplace, cu invalidare bazată pe evenimente și protecție împotriva încărcărilor bruște. A treia este evaluarea relevanței căutării, prin metrici precum `precision@k` sau `nDCG`, nu doar prin latență.

Alte îmbunătățiri țin de maturizarea operațională: limitarea ratei la autentificare, trecerea la Argon2id, metrici și tracing, paginare prin chei pentru istoricul mare, materializarea unor agregări de analiză și folosirea tabelor pregătite pentru comentarii de coach sau rezultate persistente ale analizorului. Aceste dezvoltări nu schimbă fundamentul aplicației, ci îl duc mai aproape de un produs care poate fi rulat și întreținut pe termen lung.

Bibliografie

- [1] Boyd Epley, *Poundage Chart. Boyd Epley Workout*, Lincoln, NE: Body Enterprises, 1985.
- [2] Michael B. Jones, John Bradley și Nat Sakimura, *JSON Web Token (JWT)*, RFC 7519, Internet Engineering Task Force (IETF), 2015, URL: <https://www.rfc-editor.org/rfc/rfc7519> (accesat în 16.6.2026).
- [3] OpenSearch Project, *OpenSearch Documentation*, 2024, URL: <https://opensearch.org/docs/latest/> (accesat în 16.6.2026).
- [4] Brad J. Schoenfeld, Dan Ogborn și James W. Krieger, „Dose-response Relationship Between Weekly Resistance Training Volume and Increases in Muscle Mass: A Systematic Review and Meta-analysis”, în *Journal of Sports Sciences* 35.11 (2017), pp. 1073–1082, DOI: [10.1080/02640414.2016.1210197](https://doi.org/10.1080/02640414.2016.1210197).
- [5] The PostgreSQL Global Development Group, *PostgreSQL Documentation: Full Text Search*, 2024, URL: <https://www.postgresql.org/docs/16/textsearch.html> (accesat în 16.6.2026).

Anexa 1 – Schema bazei de date

Schema este creată de migrarea Flyway V1, care definește 28 de tabele. Migrarea V2 adaugă datele oficiale pentru grupe musculare și exerciții, iar migrarea V3 adaugă indexul unic parțial care garantează că un utilizator poate avea o singură sesiune activă.

Zonă	Tabele principale
Autentificare și coaching	app_users, refresh_tokens, coach_profiles, coach_client_relationships
Catalog și planificare	muscle_groups, exercises, exercise_muscle_groups, routines, gyms, equipment
Programe	workout_templates, template_stats, template_days, template_day_exercises, template_day_exercise_muscle_groups, template_day_routines
Sesiuni istorice	workout_sessions, session_exercises, session_exercise_muscle_groups, session_routines, workout_sets
Marketplace	template_votes, template_saves, template_use_events
Extensii pregătite	coach_session_comments, coach_template_assignments, template_analysis_results, outbox_events

Tabela 1.1: Gruparea tabelor din schema PostgreSQL.

Constrângerile importante includ: valori admise pentru enumerări, days_per_week BETWEEN 1 AND 7, rpe BETWEEN 1.0 AND 10.0, finished_at >= started_at, contoare nenegative, interdicția ca un utilizator să fie propriul antrenor, publicarea doar cu published_at nenul și indecși unici parțiali pentru resursele șterse logic.

```
CREATE UNIQUE INDEX ux_workout_sessions_one_active_per_user
ON workout_sessions (user_id)
WHERE status = 'IN_PROGRESS';

CREATE UNIQUE INDEX gyms_user_name_active_idx
ON gyms (user_id, lower(name))
WHERE deleted_at IS NULL;

ALTER TABLE workout_templates
ADD CONSTRAINT check_template_visibility_published
CHECK (
  (visibility = 'PUBLIC' AND published_at IS NOT NULL)
  OR visibility = 'PRIVATE'
);
```

Anexa 2 – Listări de cod reprezentative

Următoarele fragmente sunt scurtate pentru lizibilitate, dar păstrează forma regulilor implementate în proiect.

Listarea 1. Analizorul de programe folosește reguli deterministe: fiecare sub-scor pornește de la valoarea maximă și scade penalizări fixe pentru problemele găsite.

```
private int scoreVolumeAndCoverage() {
    int score = 35;

    if (!lowerBody) {
        add("NO_LOWER_BODY", HIGH, "No lower-body work is planned.");
        score -= 12;
    }
    if (!pulling) {
        add("NO_PULL", HIGH, "No pulling movement is planned.");
        score -= 12;
    }
    if (!pushing) {
        add("NO_PUSH", HIGH, "No pushing movement is planned.");
        score -= 12;
    }

    for (String bucket : ALL_BUCKETS) {
        double volume = weeklyWeightedSets(bucket);
        if (volume <= 3) {
            add("MUSCLE_VOLUME_VERY_LOW", MEDIUM, bucket);
            score -= 3;
        } else if (volume > 25) {
            add("MUSCLE_VOLUME_EXCESSIVE", HIGH, bucket);
            score -= 5;
        } else if (volume > 20) {
            add("MUSCLE_VOLUME_HIGH", MEDIUM, bucket);
            score -= 2;
        }
    }

    return Math.max(0, score);
}
```

Listarea 2. Configurația de securitate aplică RBAC la nivel de rută; verificările de proprietate și relațiile de coaching sunt apoi aplicate în servicii.

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers(POST, "/api/auth/register", "/api/auth/login",
        "/api/auth/refresh", "/api/auth/logout").permitAll()
    .requestMatchers(GET, "/api/health").permitAll()
    .requestMatchers("/api/admin/**").hasRole("ADMIN")
    .requestMatchers("/api/coach/**").hasRole("COACH")
    .requestMatchers("/api/**").authenticated()
    .anyRequest().permitAll())
```

Listarea 3. Tabela `outbox_events` există încă din prima migrare și poate fi folosită ulterior pentru indexare tranzacțională.

```
CREATE TABLE outbox_events (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  aggregate_type varchar(100) NOT NULL,
  aggregate_id uuid NOT NULL,
  event_type   varchar(100) NOT NULL,
  payload      jsonb NOT NULL DEFAULT '{}'::jsonb,
  status       varchar(30) NOT NULL DEFAULT 'PENDING',
  attempts     int NOT NULL DEFAULT 0,
  available_at timestampz NOT NULL DEFAULT now(),
  processed_at timestampz,
  created_at   timestampz NOT NULL DEFAULT now(),
  last_error   text,
  CONSTRAINT check_outbox_status
    CHECK (status IN ('PENDING', 'PROCESSING', 'PROCESSED', 'FAILED'))
);

CREATE INDEX outbox_events_processing_idx
  ON outbox_events (status, available_at, created_at);
```

Anexa 3 – Rezultatele complete ale evaluării

Măsurătorile au fost făcute la șapte dimensiuni ale corpusului, de la 1.000 la 2.000.000 de documente. Pentru PostgreSQL s-au folosit `tsvector`, `GIN`, `ts_rank_cd` și `pg_trgm`. Pentru OpenSearch s-au folosit aceleași câmpuri denormalizate ca în indexul aplicației. Cifrele reprezintă latențe în milisecunde.

Corpus	Construire PG	Construire OS	Dimensiune PG	Dimensiune OS
1.000	216 ms	203 ms	1,6 MB	0,3 MB
10.000	1.926 ms	975 ms	14,6 MB	2,9 MB
50.000	7.966 ms	3.481 ms	68,7 MB	14,3 MB
100.000	14.006 ms	6.349 ms	135,5 MB	28,5 MB
500.000	68.752 ms	37.987 ms	664,7 MB	140,8 MB
1.000.000	125.774 ms	57.701 ms	1.313,6 MB	349,1 MB
2.000.000	239.192 ms	114.908 ms	2.632,5 MB	738,7 MB

Tabela 3.1: Timpul de construire a indexului și dimensiunea pe disc.

Anexa 4 – Contractul API (selecție)

Toate endpoint-urile, cu excepția autentificării și a verificării de sănătate, cer un token JWT valid. Erorile folosesc un format uniform, cu un cod stabil, de exemplu `TEMPLATE_NOT_FOUND`, `ACTIVE_WORKOUT_EXISTS` sau `SEARCH_UNAVAILABLE`.

Corpus	PostgreSQL ($p_{50}/p_{95}/p_{99}$)	OpenSearch ($p_{50}/p_{95}/p_{99}$)
1.000	1,46 / 2,68 / 3,12	3,25 / 5,70 / 10,56
10.000	10,64 / 14,32 / 17,63	2,85 / 5,21 / 7,17
50.000	48,70 / 66,59 / 77,97	5,17 / 10,05 / 13,98
100.000	100,21 / 117,87 / 136,46	6,84 / 10,59 / 12,83
500.000	779,47 / 1021,25 / 1229,16	22,75 / 38,09 / 67,26
1.000.000	872,19 / 1199,43 / 1220,09	33,64 / 45,24 / 83,90
2.000.000	1863,97 / 2783,12 / 2783,12	63,65 / 73,05 / 111,67

Tabela 3.2: Latența căutării full-text pentru interogarea *bench press*.

Corpus	PostgreSQL ($p_{50}/p_{95}/p_{99}$)	OpenSearch ($p_{50}/p_{95}/p_{99}$)
1.000	14,45 / 18,91 / 21,69	2,37 / 3,74 / 5,63
10.000	127,93 / 134,06 / 136,22	3,23 / 7,38 / 10,94
50.000	621,74 / 645,63 / 646,08	2,66 / 4,95 / 6,59
100.000	1831,47 / 2169,61 / 2169,61	3,15 / 5,74 / 7,38
500.000	2447,22 / 8354,43 / 8354,43	4,06 / 7,20 / 8,95
1.000.000	7806,25 / 17790,10 / 17790,10	3,94 / 9,18 / 13,35
2.000.000	15825,49 / 16399,71 / 16399,71	3,89 / 9,38 / 14,14

Tabela 3.3: Latența căutării fuzzy pentru interogarea greșită *benhc*.

```

Autentificare : POST /api/auth/register
                POST /api/auth/login
                POST /api/auth/refresh
                POST /api/auth/logout
                GET  /api/auth/me

Planificare   : /api/exercises
                /api/routines
                /api/gyms
                /api/gyms/{id}/equipment
                /api/equipment/{id}

Programe      : /api/templates
                /api/templates/{id}
                POST /api/templates/{id}/publish
                POST /api/templates/{id}/unpublish

Sesiuni       : POST /api/workouts/start
                GET  /api/workouts/active
                POST /api/workouts/{id}/finish
                POST /api/workouts/{id}/cancel
                POST /api/session-exercises/{id}/sets
                PUT|DELETE /api/sets/{id}

Istoric       : GET /api/history

```

	GET /api/analytics/overview
Marketplace	: GET /api/marketplace/templates POST /api/marketplace/templates/{id}/vote POST /api/marketplace/templates/{id}/save POST /api/marketplace/templates/{id}/use
Analizor	: GET /api/templates/{id}/analysis
Cautare	: GET /api/search/templates GET /api/search/workouts POST /api/admin/search/reindex
Coaching	: /api/coach/clients/{id}/history /api/coach/clients/{id}/analytics /api/coach/clients/{id}/sessions /api/coach/clients/{id}/search /api/coach/clients/invite DELETE /api/coach/clients/{id} /api/coaching/invites /api/coaching/coaches

Anexa 5 – Configurație și rulare

Fișierul `docker-compose.yml` pornește PostgreSQL 16, OpenSearch 2.13.0 și aplicația Spring Boot. Imaginea aplicației construiește frontend-ul Vite în timpul build-ului Maven și îl include în jar-ul Spring Boot, astfel încât interfața și API-ul sunt servite de aceeași aplicație la `http://localhost:8080`.

```
docker compose up --build
```

Configurația principală păstrează schema sub controlul Flyway și dezactivează Hibernate DDL. Modulul de căutare este dezactivat implicit și este activat în compose prin `APP_SEARCH_ENABLED=true`.

```
spring:
  jpa:
    hibernate:
      ddl-auto: validate
  flyway:
    enabled: true

app:
  security:
    jwt:
      issuer: workout-thesis
      access-ttl: 15m
```

```
    refresh-ttl: 14d
search:
  enabled: ${APP_SEARCH_ENABLED:false}
  uri: ${APP_SEARCH_URI:http://localhost:9200}
  templates-index: templates_v1
  templates-alias: templates
  sessions-index: workout_sessions_v1
  sessions-alias: workout_sessions
```

Pentru dezvoltare, frontend-ul poate rula separat prin Vite pe portul 5173, iar backend-ul pe portul 8080. Testele de integrare folosesc Testcontainers și au nevoie de Docker funcțional, deoarece pornesc PostgreSQL 16 și, pentru suita de căutare, un container OpenSearch 2.x.